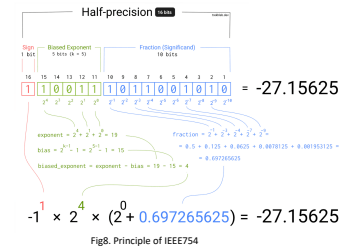
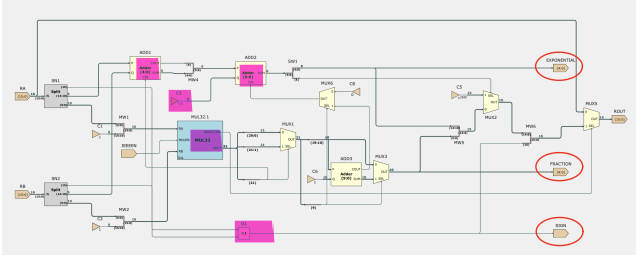


5.4 Optional Collaboration Challenge: IEEE 754 half-precision (binary16) Floating Point

IEEE-754 binary16 has structure of 3 parts: sign, fraction, and exponent



In a multiplier, we do these 3 parts separately:
For sign bit, we just do an XOR gate to get result's sign.
For exponential bits, we add them together.
For fraction bits, we multiply them.



For fraction:
Initially it is a 10-bit binary representing 0.xxxxx, we add 1 to the MSB to let it become 1.xxxxx and do a multiplication.

Solution is a 22-bit binary, and 21:20 determines whether it is 1.xxxxx or 2.xxxxx
If 21th bit = 0, there isn't carryout, so we take 19:10 and that's result.
If 21th bit = 1, there's carryout, we need to do normalization, which is LSR result by 1 bit and add 1 to exponential.

I do an easy rounding:
If 9th bit, which is MSB we discard, = 0, we round down.
If 9th bit = 1, we round up.

It is not as accurate as IEEE754 level rounding as it doesn't care about subnormal or special cases, but it will be more accurate than before.



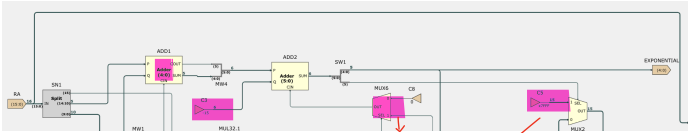
This is 9th bit

For exponential:
Because IEEE754 uses a biased exponential:

To make it possible to have a negative exponent, the IEEE 754 standard uses the **biased exponent**. The idea is simple - subtract the bias from the exponent value to make it negative. For example, if the exponent has 5 bits, it might take the values from the range of [0, 31] (all values are positive here). But if we subtract the value of 15 from it, the range will be [-15, 16]. The number 15 is called bias, and it is being calculated by the following formula:

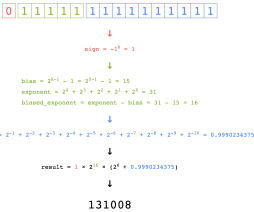
So exponential = actual exponential + 15.
If we just add them, we will get:
E(result) = actual E(result) + 15 from RA + 15 from RB.
So we have to -15

And also, remember to add the normalization result from fraction.



Due to normalization

If exponential has a carryout, it means that we are exceeding the maximum value that a 16-bit IEEE754 binary can represent, in this case I just write the largest value I can think of, x7FFF to result. And it is NaN in a standard IEEE754



Sign: XOR gate.



We have implemented this hardware, and now we can load it into DATAPATH:

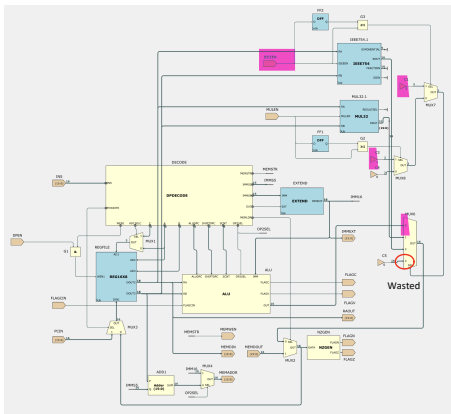


Fig13. DATAPATH with IEEE754

Creating and controlling an IEEE754 is the same as MUL32, except that this time we give it to MOV32 assembly language. Because we need just the same cycles for IEEE754 as MUL32

I use a 4-MUX to select results from different components:

0. ALU
1. MUL32
2. IEEE754
3. Wasted

We can see that in CONTROLPATH logic is same, this time we compare 4:2 to 2, which stands for MOV32

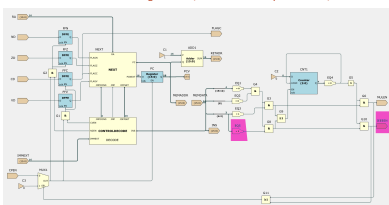


Fig14. CONTROLPATH with IEEE754

That's all for IEEE754 and we run a test:

Use website: <https://trekhlieb.dev/blog/2021/binary-floating-point/> to convert IEEE754 to decimal.

```

EXT #b57
ADD R0, #b57C //Coding 0x57BC, which is 123.75 in R0

EXT #b51
ADD R1, #b51 //Coding 0x51B0, which is 45.5

EXT #bCA
ADD R2, #bCA //Coding 0xCA00, which is -13.25 in R2

EXT #b3E
ADD R3, #b3E //Coding 0x3E00, which is 1.5

MOV32 R0, R1
//Run IEEE754, expected = 5638.625
MOV32 R2, R3
//Expected = -19.875
    
```

Fig15. Test

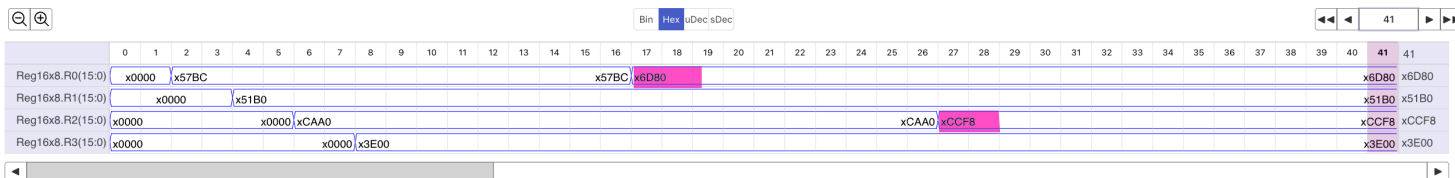


Fig16. Test result

We get x6D80 and xCCF8:

0 1 1 0 1 1 0 1 1 0 0 0 0 0 0 0

$$\text{sign} = -1^0 = 1$$

$$\begin{aligned} \text{bias} &= 2^{k-1} - 1 = 2^{5-1} - 1 = 15 \\ \text{exponent} &= 2^4 + 2^3 + 2^1 + 2^0 = 27 \\ \text{biased_exponent} &= \text{exponent} - \text{bias} = 27 - 15 = 12 \end{aligned}$$

$$\text{fraction} = 2^{-2} + 2^{-3} = 0.375$$

$$\text{result} = 1 \times 2^{12} \times (2^0 + 0.375)$$

1 1 0 0 1 1 0 0 1 1 1 1 1 0 0 0

$$\text{sign} = -1^1 = -1$$

$$\begin{aligned} \text{bias} &= 2^{k-1} - 1 = 2^{5-1} - 1 = 15 \\ \text{exponent} &= 2^4 + 2^2 + 2^0 = 19 \\ \text{biased_exponent} &= \text{exponent} - \text{bias} = 19 - 15 = 4 \end{aligned}$$

$$\text{fraction} = 2^{-3} + 2^{-4} + 2^{-5} + 2^{-6} + 2^{-7} = 0.2421875$$

$$\text{result} = -1 \times 2^4 \times (2^0 + 0.2421875)$$

5632 (3 sf)

-19.875

Which is quite close and acceptable.

Statistic:

1. Adders used: 53 (could be 43 because a 22-bit multiplier would be enough but I am lazy so use MUL32 in last section).
2. Speed: 10 cycles.
3. Can do IEEE754 multiplication to a certain, and acceptable accuracy, but cannot deal with subnormal or special cases.

Subnormal is an IEEE-754 floating-point number that is very close to zero but not equal to zero. It fills the gap between the smallest normalised number and zero, so results don't abruptly underflow straight to 0.
Which is Exponential = 0 && Fraction != 0.